App Dev Project Report

1. Student Details

Name: Jayesh Bansal
Roll Number: 24f1001895
Email: 24f1001895@ds.study.iitm.ac.in
About Me: I am a Data Science undergraduate at IIT Madras BS Degree program with a deep interest in web application development and data-driven technologies. I enjoy building meaningful applications that combine learning, analytics, and user experience.

---

2. Project Details

Project Title: Hospital Management System (HMS)

Problem Statement:

A comprehensive Hospital Management System designed to streamline healthcare operations. It facilitates role-based interactions for Admins, Doctors, and Patients, covering appointment scheduling, medical record management, and automated reporting.

Approach:

The application implements a decoupled Client-Server Architecture.

- Backend: Built using Flask to serve as a RESTful API. It handles business logic, database transactions, and asynchronous background tasks.
- Frontend: Built using Vue.js (CLI) to provide a Single Page Application (SPA) experience. It communicates with the backend via JSON to render dynamic content without page reloads.
- Optimization: To ensure performance under load, Redis was integrated for caching frequently accessed data (like parking availability), and Celery was used to offload time-consuming tasks like generating PDF reports and sending emails.

---

3. AI/LLM Declaration

I used ChatGPT (GPT-5) and GitHub Copilot to assist in the development process.

● Usage Areas:

- Debugging complex SQLAlchemy relationships and cascade delete behaviors.
- Generating boilerplate code for Vue.js components and Vuex state management.
- Refactoring CSS into BootstrapVue components for consistency.

AI LLM Usage Overall Summary:

Category	Percentage Used	Notes/Justification
Backend (Flask API, DB, Celery)	**26%**	Used to debug complex SQLAlchemy queries for the analytics dashboard and to understand the configuration syntax for Redis and Celery workers. Core routing and logic were written manually.
Frontend (Vue.js, Vuex, Axios)	**35%**	Used to understand Vue lifecycle hooks (e.g., mounted vs created) and to generate boilerplate code for Vuex state management. Specific component logic and API integration were manual.
Styling / UI (Bootstrap 5)	**3%**	Minimal usage. Used mainly to quickly generate responsive grid layouts (Cards/Modals) using Bootstrap 5 syntax, which I then customized.
Auth & Business Logic (JWT)	**10%**	Assistance restricted to understanding the implementation of flask-jwt-extended for token-based authentication (refresh tokens/protection).

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
Flask-Login	User authentication and session management

Celery + Redis	Asynchronous task queue for handling background jobs like email reminders and report generation
Flask-JWT-Extended	Secure JSON Web Token authentication for stateless user sessions.
SQLite	Lightweight local database for storing user data

---

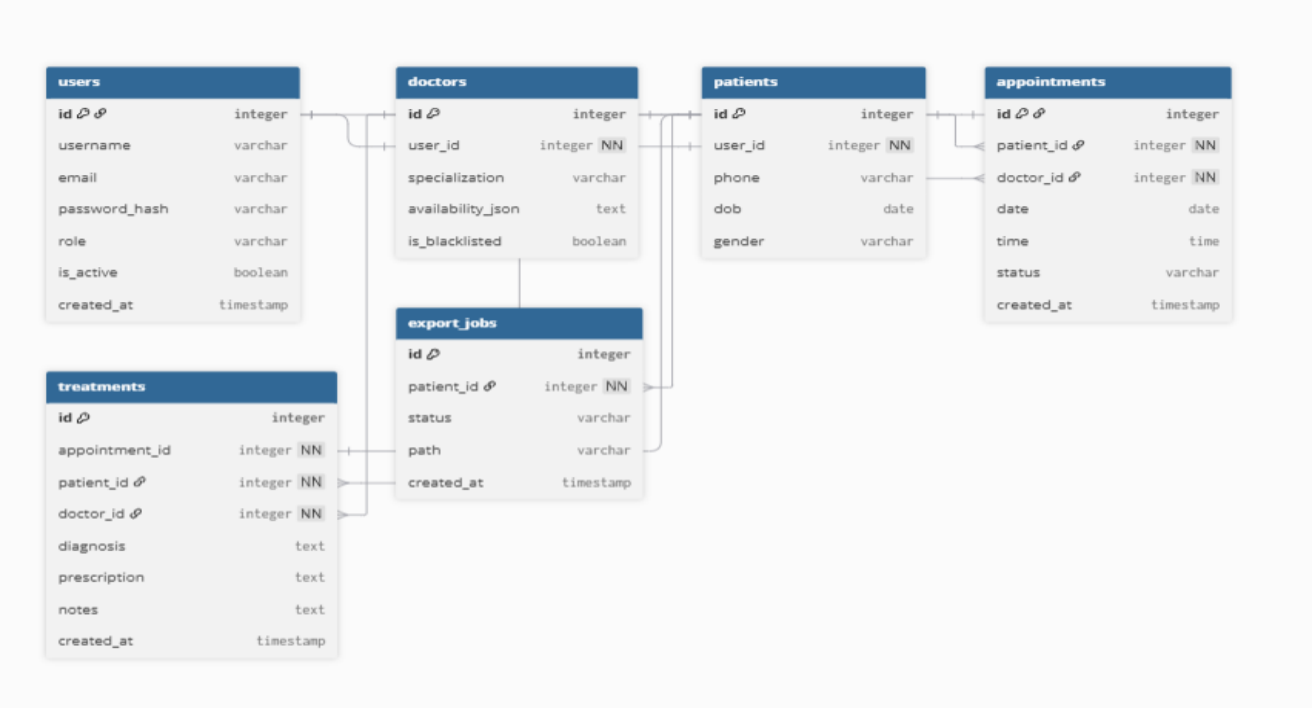
5. Database Schema / ER Diagram

Tables:

1. **User**: Central authentication entity.
2. **Doctor**: Extends User profile.
3. **Patient**: Extends User profile.
4. **Appointment**: Links Patient and Doctor.
5. **Treatment**: Medical record linked to an appointment.
6. **ExportJob**: Async job tracking.

Relationships:

- **User** ↔ **Doctor**: One-to-One
- **User** ↔ **Patient**: One-to-One
- **Doctor** ↔ **Appointment**: One-to-Many
- **Patient** ↔ **Appointment**: One-to-Many
- **Appointment** ↔ **Treatment**: One-to-One
- **Patient** ↔ **Treatment**: One-to-Many
- **Doctor** ↔ **Treatment**: One-to-Many



7. API Resource Endpoints

Endpoint	Method	Description
/api/auth/login	**POST**	Authenticate user and return JWT token
/api/auth/register	**POST**	Register a new patient account
/api/auth/profile	**GET**	Fetch current user profile details
/api/auth/profile	**PUT**	Update user profile information
/api/admin/users	**GET**	Get system statistics (counts)
/api/admin/doctors	**POST**	Create a new doctor account
/api/admin/doctors/{id}	**PUT**	Update doctor details (availability, blacklist)
/api/admin/patients/{id}/blacklist	**POST**	Blacklist a patient
/api/admin/patients/{id}/unblacklist	**POST**	Unblacklist a patient
/api/admin/trigger_reminders	**POST**	Manually trigger daily email reminders
/api/admin/trigger_monthly_reports	**POST**	Manually trigger monthly report generation
/api/doctor/dashboard	**GET**	Fetch doctor's appointments and availability
/api/doctor/set_availability	**POST**	Update doctor's availability schedule
/api/doctor/appointments/{id}/treatment	**PUT**	Add/Update diagnosis and prescription
/api/patient/search_doctors	**GET**	Search doctors by name or specialization
/api/patient/history	**GET**	Fetch patient's treatment history
/api/appointments/slots	**GET**	Get available time slots for a doctor

/api/appointments/book	**POST**	Book a new appointment
/api/appointments/{id}/reschedule	**POST**	Reschedule an existing appointment
/api/appointments/{id}/cancel	**POST**	Cancel an appointment
/api/appointments/my	**GET**	Fetch current user's appointments
/api/appointments/export_my_treatments	**POST**	Request CSV export of treatment history
/api/appointments/export_download/{job_id}	**GET**	Download the exported CSV file

---

7. Architecture and Features (optional)

Architecture Overview:

Component	Key Locations	Description
Backend API	**backend/app/routes/**	Flask application organized using Blueprints for modularity.
Database Layer	**backend/app/models.py**	SQLAlchemy ORM for schema definition and database interactions.
Async Tasks	**backend/app/tasks.py**	Celery + Redis for background processing (emails, reports).
Frontend UI	**frontend/src/views/**	Vue.js 3 Single Page Application (SPA) with Bootstrap 5.
Authentication	**backend/app/routes/auth.py**	JWT (JSON Web Token) for secure, stateless user sessions.
Configuration	**backend/config.py, .env**	Environment-based config for DB, Mail, and Keys.

Implemented Features:

- **Role-Based Access Control**: Secure login for Admins, Doctors, and Patients.
- **Appointment Management**: Booking, rescheduling, and cancellation with strict availability checks.
- **Medical Records**: Doctors can record diagnosis and prescriptions; Patients can view history.
- **Automated Reminders**: Daily email reminders for appointments sent via Celery.
- **Reporting**: Monthly HTML activity reports for doctors and CSV export of treatment history for patients.

- **Doctor Availability**: JSON-based flexible availability scheduling.
- **Admin Dashboard**: User management, blacklisting, and system statistics.

---

8. Video Presentation

Drive Link:

https://drive.google.com/file/d/1_WdI75qEmCkS-EE5pIOdQ9aiyDuvpreY/view?usp=sharing